定義

Vibe Coding＝日本語で頼み、結果を見て進めるスタイル

Andrej Karpathy が名付けた（2025年2月、Xへの投稿）

- 雰囲気（vibe）に身をまかせ、コードの存在すら忘れて進める
- AIが提案する変更を、細かく読まずに「すべて受け入れる」
- エラーが出たら、メッセージをそのままAIに貼り付ける
- コードが自分の理解を超えて育っても、先へ進む

出典の要旨。教科書 第1章 1.1

手に入るもの

速い・始めやすい・対話できる

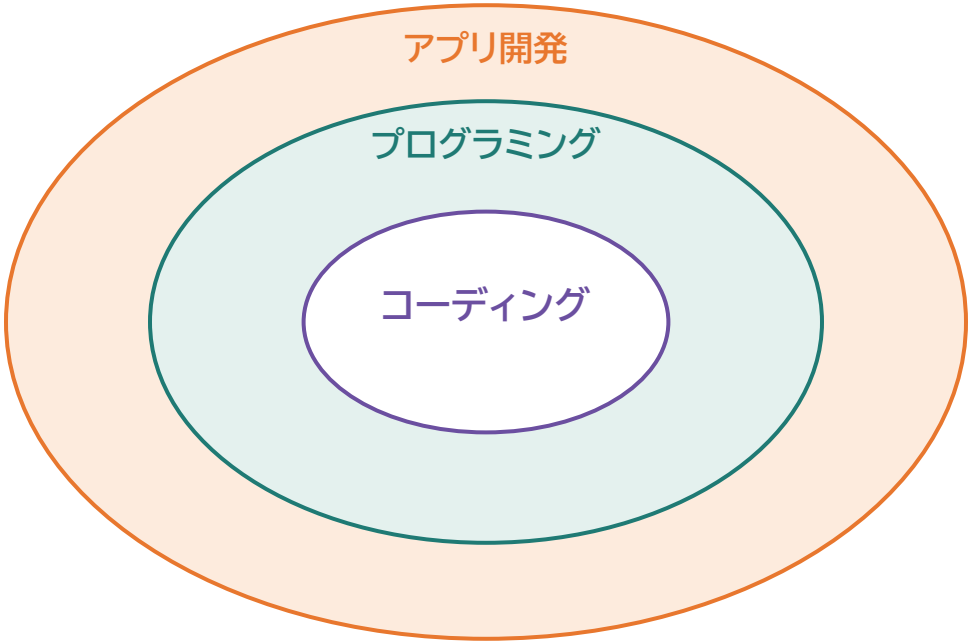
後回しにしているもの

コードの理解・変更内容の確認・検証と対応準備

速さと引き換えに、理解と確認を後回しにする。それがこのスタイルの正直な設計

どこまでAIに任せる？

コーディング ⊂ プログラミング ⊂ アプリ開発



コーディング

決まった内容をコードに書く（清書）

AIが強く助ける

プログラミング

手順を考え、組み立て、動かして確かめる

AIがかなり助ける

アプリ開発

誰の困りごとを解くか決め、確かめ、届け、使われ続ける状態を保つ

人の判断が増える

今日のデモで触れるのは、いちばん内側だけ

ライブデモ

DEMO②　言葉で変える。具体的に頼むか、曖昧に頼むか

具体的な頼み方
「つかれた」のボタンだけ紫色にしてください。あと、記録の文章を「17:03 つかれた」のように短くしてください。

→ 結果を見比べやすい

曖昧な頼み方
全体を、もっとやさしい雰囲気にしてください。

→ 色を淡く？角を丸く？　人によって解釈が分かれる

曖昧な部分は、AIが「仮定」で埋める。違っていたら言い直せばよい。変えたら、色だけでなく「記録が増える」ことも確かめる

頼み方の型

壊れの連鎖を止める3つの鉄則

まず計画を出させる

「まず計画だけ出してください。実装はまだしないでください」

作る前なら、ずれを言葉の段階で直せる

1回の依頼＝1つの変化

「取り消しを直して、ついでに集計も、あと色も」は3つの変化

完成条件が1〜2行で書けないタスクは、まだ大きすぎる

仮定と不明点を挙げさせる

「実装の前に、あなたが置く仮定と、決まっていないことを列挙して」

答えた内容は仕様書に書き戻す

直らないときは同じ指示を繰り返さない：①言い方を変える（事実を書く）②小さくする ③戻す

困ったとき

エラーが出たら／AIに自分でテストさせる

エラーの伝え方

- 何をしたら、何が起きたか（操作と結果）
- エラー文は要約せず、全文を貼る
- 貼る前に、APIキー・パスワード・個人情報を伏せる（例：apiKey=[伏せた]）
- 自動修正に任せたら「何が原因だったか」も聞く

役割を分ける

実装役 ブリーフ通りに作る

テスト役 壊すつもりで確かめる（検算・意地悪な入力）

レビュー役 仕様と比べ、漏れと余計な変更を探す

AIの「確認しました」は証拠ではない。実行した方法と結果を出させ、自分の画面でも試す